

HW2 – Group5

Calabrese Lorenzo (s345902) Calfapietro Flavia (s345862) Spedicato Giuseppe (s337116)

1. Methodology

The KWS pipeline was developed by exploring multiple configurations of the audio front-end based on MFCC features, varying frame length, frame step, number of Mel bins and MFCC coefficients. For each configuration, the dataset was preprocessed and passed through an oversized CNN. The architecture was then refined using PIT (Pruning-Inspired Technique), a training strategy that jointly optimizes accuracy and model complexity via channel-level masking. The pruned network was exported and fine-tuned to recover accuracy degradation introduced by pruning. Each optimized model was evaluated on the test set and exported in ONNX format, recording its final size and accuracy. A static INT8 quantization step was applied using ONNX Runtime to reduce memory footprint and inference latency.

2. Feature Extraction

The frame length was set to 16 ms, corresponding to exactly 256 samples. This power-of-two alignment maximizes FFT computational efficiency and eliminates the need for zero-padding. For the frame step, a value of 12 ms was chosen. This introduces a temporal overlap that enhances resolution, effectively compensating for the information loss caused by the aggressive reduction of the feature set (12 Mels and 8 MFCCs) required to minimize input dimensionality and MAC operations. Finally, a high-pass filter with $f_{\min} = 50$ Hz was applied to suppress low-frequency noise, ensuring that the limited model capacity is focused exclusively on relevant speech components.

Hyperparameter	Value
frame_length_in_s	0.016
frame_step_in_s	0.012
n_mels	12
f_min	50
f_max	8000
n_mfcc	8

Table 1: Feature extraction hyperparameters optimized for latency.

3. Training Hyperparameters

Training is performed in two phases: joint optimization of model weights and NAS masks, followed by fine-tuning of the final architecture. The model is trained for 3000 steps using mini-batches of size 32. After 20% of the training steps, NAS parameters are optimized using the Plinio library, which applies node-based pruning by minimizing the validation loss com-

Hyperparam.	Value
train_steps	3000
train_batch_size	32
fine_tune_steps	1000
reg_strength	2×10^{-6}

Table 2: Training hyperparameters.

bined with a complexity regularizer set to 2×10^{-6} . This specific value is critical as it controls the trade-off between sparsity and accuracy, imposing a stricter penalty on model complexity to prevent overfitting on the compact 8-MFCC input. Once the optimal mask configuration is learned, the pruned model is exported and fine-tuned for 1000 additional steps. This phase is essential to allow the remaining weights to adapt to the pruned structure and recover any accuracy lost due to the aggressive feature reduction.

4. Model Design & Optimization

The final model is a compact CNN designed for low-latency keyword spotting. It begins with a strided convolution for initial temporal downsampling, reducing the computational load for subsequent layers. The feature extraction backbone alternates standard 3×3 convolutions (to capture local patterns) and 1×1 convolutions (to mix channel information), all followed by Batch Normalization and ReLU activations. A Global Average Pooling layer aggregates the features into a fixed-size vector, removing the dependency on input resolution and feeding directly into a lightweight fully connected classifier. Model complexity is further optimized through NAS-driven node-based pruning, while static INT8 quantization is applied after ONNX export to minimize size and latency.

5. Results & Discussion

Acc. (%)	Size (MB)	Lat. (ms)
99.5	0.218	3.1

Table 3: Evaluation results of the KWS pipeline.

The model achieves 99.5% accuracy while maintaining a median latency of 3.1 ms, confirming that the system is optimized for real-time inference. The reduction in latency is primarily due to the power-of-two FFT configuration and the reduced number of MFCC features, while increased training and fine-tuning compensate for the more compact input representation. These results demonstrate the effectiveness of combining pruning and quantization to obtain an efficient yet accurate embedded KWS solution.